

Design-Based Latent Class Analysis for Complex Survey Data

Methodology and worked application: institutional trust, LAPOP AmericasBarometer Mexico 2023

Kevin Linares

2026-07-29

Table of contents

1	Purpose and scope	2
2	Where this sits in the Total Survey Error framework	2
3	Data and preparation	3
4	The latent class model	8
5	Design-weighted estimation	8
6	Model selection	9
7	The chosen model	11
8	Design-based variance estimation	12
9	Fit diagnostics	14
10	Validation against an independent implementation	18
11	Naming the segments	19
12	The measurement model	22
13	Scoring segment membership	24
14	Domain estimation	26
14.1	age_cat	28
14.2	sex	30
14.3	education	31
14.4	urban	32
14.5	employment	33

15 The delivered data product	34
16 Limitations	34
17 Replication	35
References	36

1 Purpose and scope

This document is both the methodology of a design-based latent class analysis (LCA) workflow and a worked application of it. Every estimator is stated where it is first used and immediately applied, so a reader can follow the reasoning and the results together. The application is the institutional trust battery from the 2023 LAPOP AmericasBarometer survey of Mexico; the survey is an example, and the methods are the subject.

Throughout, a latent class is called a **segment**. This is a word-choice preference for applied audiences; the statistical object is unchanged.

The workflow’s organizing commitment is that **one replicate design generates every standard error in the analysis**. The same stratified jackknife replicate weights that produce the interval on a conditional response probability produce the interval on a segment prevalence within a demographic group. Nothing is computed under an independence assumption at any stage.

The closest existing software is the `baysc` package (Wu et al. 2024, 2026), which fits a weighted objective within a Bayesian pseudo-posterior with a post-hoc variance adjustment. It differs in three respects: it selects the number of segments automatically through an overfitted mixture, it obtains uncertainty from an adjusted posterior rather than replicate weights, and being Bayesian it propagates classification uncertainty through the posterior draws, which makes the attenuation discussed in Section 14 a non-issue in that framework. The two are complements. This workflow is the frequentist, replicate-variance counterpart, and it has base survey dependencies only, which matters in analytic environments restricted to CRAN.

Only two files change between applications. `survey_data_config.R` reads and cleans one dataset and defines `cfg`; `survey_lca_source.R` holds the estimation engine and is never edited per dataset. Section 17 documents what a port requires.

2 Where this sits in the Total Survey Error framework

The Total Survey Error (TSE) framework enumerates the distinct sources of error that arise as a survey is conducted, organized around the survey life cycle and split into two arms (Groves et al. 2009; Groves and Lyberg 2010). The **representation** arm runs from the target population through coverage error to the sampling frame, sampling error to the sample, nonresponse error to the respondents, and adjustment error to the survey statistic. The **measurement** arm runs from the construct through validity error to the measurement, measurement error to the response, and processing error to the edited data. Errors attach to *transitions*, not to the survey as a whole, which is why several are properties of an estimate rather than of the survey.

Locating this workflow in that framework clarifies what it does and does not accomplish.

Sampling error is what this workflow is chiefly about. Sampling error is the one representation error that is intended and controllable by design, and under a sound design it is variance rather than bias. The difficulty is that estimating it correctly requires using the realized design, and a recognized weakness of practice is that many organizations report only sampling error while computing it as though the sample were simple and random. A latent class model fit to a stratified, clustered sample and analyzed with default software produces margins of error that are too narrow, and the understatement can be large. Section 8 addresses this directly.

Adjustment error is engaged but not removed. Post-survey weighting reduces nonresponse and noncoverage bias at the cost of increased sampling variance, and that increase is itself a source of error. This workflow carries the released weights into every step of estimation so that segments describe the population, and reports the unequal weighting effect so the variance cost is visible rather than implicit.

Nonresponse error is addressed at the item level only. Respondents who skipped some items are scored from the items they answered rather than discarded, and all domain estimation runs on that larger frame, because item nonresponse on attitude batteries is not random (Section 13). Unit nonresponse is handled by the data provider’s weights and is not modeled here.

Measurement error is the reason to use a latent class model at all. LCA is used in survey research precisely because it accommodates error-prone categorical indicators: the latent segment is the construct, the items are its fallible measurements, and the conditional response probabilities are estimates of how the measurement behaves within each segment. The model does not remove measurement error; it makes it a parameter.

Processing error is introduced by this workflow and must be acknowledged. Two steps add error after the response is recorded. Assigning each respondent to a single most likely segment introduces classification error, which attenuates associations between segments and covariates (Section 14). Drafting segment names with a language model is a processing step whose output is a label, reviewed and owned by the analyst, that never enters a computation (Section 11).

Coverage error and validity are out of scope. They are properties of the frame and the questionnaire, fixed before any analysis begins.

Two caveats about the framework itself are worth carrying forward. TSE centers on accuracy and is largely silent on the other dimensions of data quality: relevance, timeliness, accessibility, interpretability, and coherence (Brackstone 1999). And its tree structure can imply that error sources are independent when they are frequently correlated; the classification error introduced in Section 14, for example, is larger for respondents who answered fewer items, which links a processing error to an item nonresponse error.

3 Data and preparation

The 2023 AmericasBarometer round in Mexico interviewed voting-age adults under a stratified, clustered design with released base weights (LAPOP Lab 2023). We analyze the thirteen-item institutional trust battery, each item answered on a seven-point scale anchored at 1 (*nada*, none at all) and 7 (*mucho*, a lot).

Preparation happens entirely in `survey_data_config.R` and follows three rules. Nonresponse codes are recoded to missing, with the source file read so those codes are visible rather than silently converted on import. Items are recoded to consecutive integers over their observed categories on *every* row of the file, so the estimation frame and the prediction frame of Section 13 share one coding by construction. Demographic recodes are written as one exhaustive `recode_values` per variable with `unmatched = "error"`, so the render halts if any source label falls through, since a pattern-matching scheme can silently misclassify or delete respondents.

```
plot_item_stack(survey_dat_full, items, "Items as read, with missingness")
```

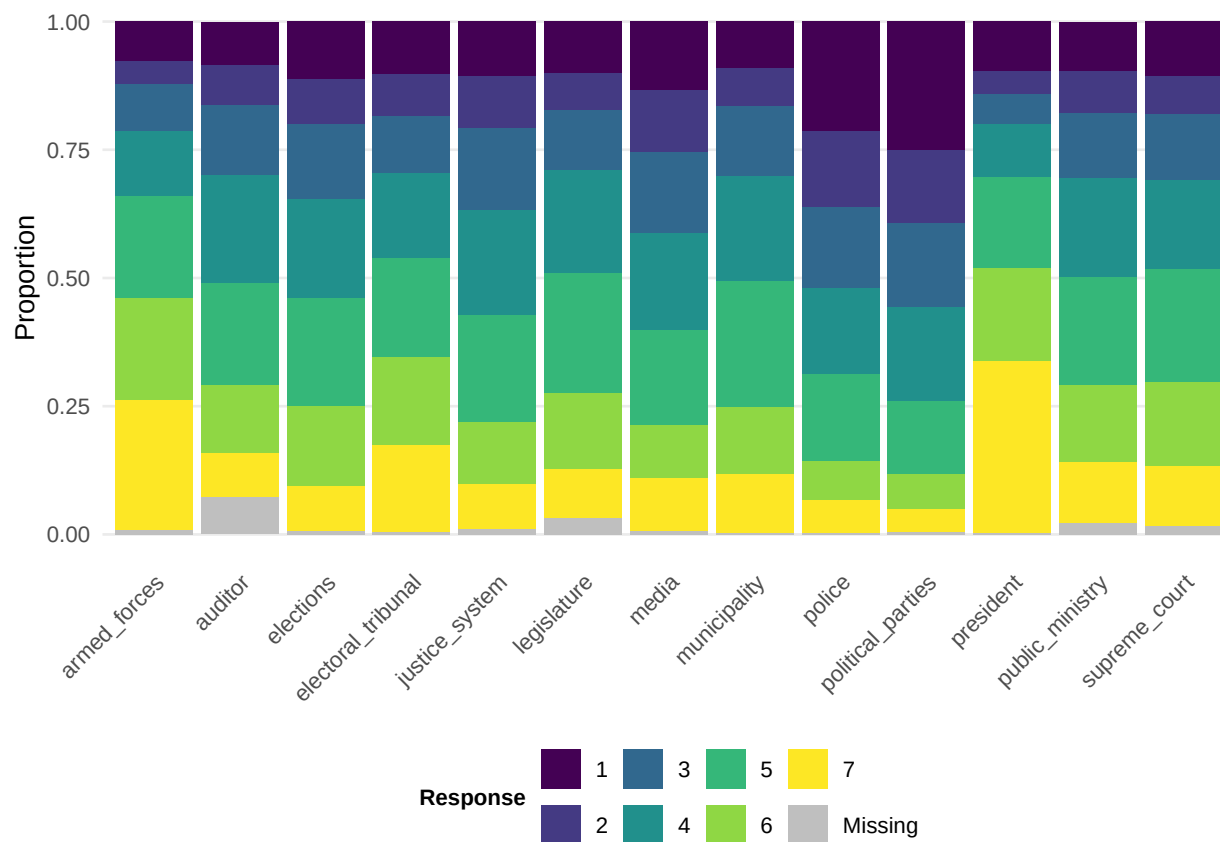


Figure 1: All respondents, all items, with missingness shown explicitly in grey. The analysis rule removes the grey.

```
cat(sprintf("%d of %d respondents enter estimation.\n",
            nrow(cfg$data), nrow(survey_dat_full)))
```

1454 of 1622 respondents enter estimation.

```
lca_table(recode_audit,
          col.names = c("Variable", "Source label", "Recoded to", "n"),
```

```
widths = c("6em", "21em", "9em", "3em"),
caption = "Recode audit: what every source label became. An unmatched label would have halted the render, so this table is confirmation rather than a search."
```

Table 1: Recode audit: what every source label became. An unmatched label would have halted the render, so this table is confirmation rather than a search.

Variable	Source label	Recoded to	n
age_cat	29	18-29	70
age_cat	18	18-29	69
age_cat	28	18-29	67
age_cat	50	45-59	63
age_cat	27	18-29	53
age_cat	48	45-59	49
age_cat	19	18-29	42
age_cat	26	18-29	41
age_cat	52	45-59	41
age_cat	49	45-59	39
age_cat	32	30-44	37
age_cat	25	18-29	36
age_cat	24	18-29	33
age_cat	40	30-44	33
age_cat	31	30-44	32
age_cat	35	30-44	31
age_cat	51	45-59	31
age_cat	30	30-44	30
age_cat	22	18-29	29
age_cat	42	30-44	29
age_cat	53	45-59	29
age_cat	20	18-29	28
age_cat	23	18-29	27
age_cat	43	30-44	27
age_cat	33	30-44	25
age_cat	37	30-44	25
age_cat	39	30-44	25
age_cat	44	30-44	25
age_cat	46	45-59	25
age_cat	54	45-59	25
age_cat	21	18-29	24
age_cat	36	30-44	24
age_cat	45	45-59	23
age_cat	38	30-44	22
age_cat	60	60+	22
age_cat	47	45-59	21
age_cat	55	45-59	21
age_cat	56	45-59	21
age_cat	34	30-44	20
age_cat	72	60+	20
age_cat	63	60+	19
age_cat	70	60+	19
age_cat	61	60+	18
age_cat	58	45-59	17
age_cat	62	60+	17
age_cat	68	60+	17
age_cat	57	45-59	15
age_cat	66	60+	15
age_cat	41	30-44	14
age_cat	65	60+	14
age_cat	69	60+	14
age_cat	59	45-59	13
age_cat	64	60+	13
age_cat	73	60+	13

Table 1: Recode audit: what every source label became. An unmatched label would have halted the render, so this table is confirmation rather than a search. *(continued)*

Variable	Source label	Recoded to	n
age_cat	67	60+	11
age_cat	78	60+	7
age_cat	71	60+	6
age_cat	77	60+	6
age_cat	74	60+	5
age_cat	76	60+	5
age_cat	82	60+	4
age_cat	85	60+	4
age_cat	75	60+	3
age_cat	79	60+	3
age_cat	80	60+	3
age_cat	81	60+	3
age_cat	84	60+	3
age_cat	83	60+	2
age_cat	87	60+	2
age_cat	86	60+	1
age_cat	92	60+	1
age_cat	94	60+	1
education	Secundaria o Educación Media Superior/Bachillerato/Preparatoria/Profesional Técnico completa	Secondary	775
education	Universitaria, superior no universitaria o técnico universitario completa	Tertiary	218
education	Primaria completa	Primary	193
education	Secundaria o Educación Media Superior/Bachillerato/Preparatoria/Profesional Técnico incompleta	Secondary	168
education	Primaria incompleta	Primary	133
education	Universitaria, superior no universitaria o técnico universitario incompleta	Tertiary	95
education	Ninguna	None	38
education	NR	NA	2
employment	Trabajando?	Employed	787
employment	Se dedica a los quehaceres de su hogar?	Homemaker	447
employment	Es estudiante?	Student	95
employment	Está jubilado, pensionado o incapacitado permanentemente para trabajar?	Retired	95
employment	Está buscando trabajo activamente?	Unemployed	73
employment	No está trabajando en este momento pero tiene trabajo?	Employed	70
employment	No trabaja y no está buscando trabajo?	Not in labor force	50
employment	NR	NA	3
employment	DK	NA	2
prez_rating	Bueno	Good	778
prez_rating	Muy bueno	Very good	394
prez_rating	Ni bueno, ni malo (regular)	Neither	335
prez_rating	Malo	Bad	66
prez_rating	Muy malo (pésimo)	Very bad	44
prez_rating	DK	NA	4
prez_rating	NR	NA	1
satis_demo	Satisfecho(a)	Satisfied	851
satis_demo	Insatisfecho(a)	Dissatisfied	529
satis_demo	Muy insatisfecho(a)	Very dissatisfied	110
satis_demo	Muy satisfecho(a)	Very satisfied	103
satis_demo	DK	NA	25
satis_demo	NR	NA	4
sex	Mujer/femenino	Female	830
sex	Hombre/masculino	Male	787
sex	DK	NA	3

Table 1: Recode audit: what every source label became. An unmatched label would have halted the render, so this table is confirmation rather than a search. *(continued)*

Variable	Source label	Recoded to	n
sex	NR	NA	2
urban	Urbano	Urban	1294
urban	Rural	Rural	328

```
dictionary |>
  mutate(responses = map_chr(responses, \(x) paste(x, collapse = " | "))) |>
  lca_table(col.names = c("Analysis name", "Source variable",
    "Survey question", "Response labels"),
    widths = c("8em", "5em", "16em", "14em"),
    caption = "Item dictionary. The response labels are in fitted-category order, and are the text the
```

Table 2: Item dictionary. The response labels are in fitted-category order, and are the text the segment labeler of Section 10 reads.

Analysis name	Source variable	Survey question	Response labels
justice_system	b10a	Confianza en el sistema de justicia	Nada 2 3 4 5 6 Mucho
electoral_tribunal	b11	Confianza en el Tribunal Supremo Electoral	Nada 2 3 4 5 6 Mucho
armed_forces	b12	Confianza en las fuerzas armadas	Nada 2 3 4 5 6 Mucho
legislature	b13	Confianza en la legislatura	Nada 2 3 4 5 6 Mucho
public_ministry	b15	Confianza en el Ministerio Público	Nada 2 3 4 5 6 Mucho
police	b18	Confianza en la policía nacional	Nada 2 3 4 5 6 Mucho
auditor	b19	Confianza en la Auditoría Superior de la Federación	Nada 2 3 4 5 6 Mucho
political_parties	b21	Confianza en los partidos políticos	Nada 2 3 4 5 6 Mucho
president	b21a	Confianza en el presidente	Nada 2 3 4 5 6 Mucho
supreme_court	b31	Confianza en el tribunal supremo	Nada 2 3 4 5 6 Mucho
municipality	b32	Confianza en su municipalidad	Nada 2 3 4 5 6 Mucho
media	b37	Confianza en los medios de comunicación	Nada 2 3 4 5 6 Mucho
elections	b47a	Confianza en las elecciones	Nada 2 3 4 5 6 Mucho

```
dat <- cfg$data
w_vec <- dat[[cfg$weight]]
n_obs <- nrow(dat)
scale_ic <- n_obs / sum(w_vec)
cv_w <- sd(w_vec) / mean(w_vec)

cat(sprintf("Weights: CV = %.3f, unequal weighting effect = %.2f.\n",
  cv_w, 1 + cv_w^2))
```

Weights: CV = 0.000, unequal weighting effect = 1.00.

The unequal weighting effect $1 + CV^2(w)$ is the multiplicative variance penalty that weight variation imposes even when the weights are unrelated to the survey variable (Kish 1965). It is the adjustment-error cost referred to in Section 2. When it is close to 1 the weighted and unweighted fits will differ little, and the comparison in Section 10 should be read accordingly.

4 The latent class model

Let $\mathbf{y}_i = (y_{i1}, \dots, y_{iJ})$ be respondent i 's answers to J categorical items, item j having C_j categories. With K segments, prevalences π_k , and conditional response probabilities $\rho_{jck} = P(y_{ij} = c \mid X_i = k)$ where X_i denotes latent membership,

$$P(\mathbf{y}_i) = \sum_{k=1}^K \pi_k \prod_{j=1}^J \rho_{j y_{ij} k}, \quad \sum_{k=1}^K \pi_k = 1, \quad \sum_{c=1}^{C_j} \rho_{jck} = 1. \quad (1)$$

The product over items is **local independence**: conditional on segment, items are independent (Lazarsfeld and Henry 1968; Goodman 1974). It is the assumption that makes the model parsimonious, that licenses scoring partially observed respondents in Section 13, and that fails first under misspecification, which is why Section 9 probes it. Model dimension is

$$d_K = (K - 1) + K \sum_{j=1}^J (C_j - 1), \quad (2)$$

one free parameter per prevalence beyond the last and $C_j - 1$ free conditional probabilities per item per segment. This grows linearly in K with slope $\sum_j (C_j - 1)$, so batteries with many response categories become expensive quickly: thirteen seven-category items at $K = 5$ carry 394 free parameters.

5 Design-weighted estimation

Under a complex design with base weights w_i , estimation maximizes the pseudo-log-likelihood

$$\ell_w(\theta) = \sum_{i=1}^n w_i \log \left(\sum_{k=1}^K \pi_k \prod_{j=1}^J \rho_{j y_{ij} k} \right), \quad (3)$$

the standard device for latent class analysis under complex sampling (Patterson, Dayton, and Graubard 2002; Vermunt and Magidson 2007; Asparouhov 2005; Skinner 1989). This is a pseudo-likelihood, not a likelihood: it is the sample-based estimate of the census log-likelihood that would be computed if the whole population were observed. Its maximizer is consistent for the census parameter under standard conditions, but it is not the maximizer of a probability model for the observed data, which is why the design must supply the variance rather than the curvature of ℓ_w .

The EM algorithm (Dempster, Laird, and Rubin 1977) alternates an expectation step computing posterior membership,

$$p_{ik} = \frac{\pi_k \prod_j \rho_{j y_{ij} k}}{\sum_{l=1}^K \pi_l \prod_j \rho_{j y_{ij} l}}, \quad (4)$$

and a maximization step in which weights enter every sufficient statistic,

$$\hat{\pi}_k = \frac{\sum_i w_i p_{ik}}{\sum_i w_i}, \quad \hat{\rho}_{jck} = \frac{\sum_i w_i p_{ik} \mathbf{1}(y_{ij} = c)}{\sum_i w_i p_{ik} \mathbf{1}(y_{ij} \text{ observed})}. \quad (5)$$

Three implementation points are load-bearing rather than incidental. Restricting the $\hat{\rho}$ denominator to respondents who answered item j is what keeps $\sum_c \hat{\rho}_{jck} = 1$ under item nonresponse; with complete cases the restriction is vacuous. The expectation step accumulates $\log \rho$ across items and applies the log-sum-exp identity $\log \sum_k e^{z_k} = z_{\max} + \log \sum_k e^{z_k - z_{\max}}$ before exponentiating, because with many items the raw product underflows to zero and Equation 4 becomes 0/0. And a missing item contributes zero on the log scale, which is exactly equivalent to dropping it from the product in Equation 1; this equivalence, not an approximation, is what Section 13 relies on.

Iteration stops when $|\Delta \ell_w| < \varepsilon(|\ell_w| + 1)$, with $\varepsilon = 10^{-8}$ during enumeration and 10^{-10} for the chosen model and every replicate refit. Matching the replicate tolerance to the point-estimate tolerance matters because the jackknife variance is a sum of squared deviations from the full-sample estimate, so systematically under-converged replicates would enter as a bias in every one of those deviations.

Because the pseudo-likelihood of a finite mixture can be multimodal with near-tied optima, each candidate model is fit from `cfg$n_starts` random starts and the reported solution is the best mode found; the claim is stated rather than hidden. Starting values are generated in the main session from a deterministic seed sequence and passed to parallel workers as data, so no worker touches the random number generator and results are identical under sequential and parallel execution. Point estimates are invariant to the scale of the weights.

The likelihood is invariant to permutation of segment labels, so any two fits may agree completely and still not be comparable coordinate by coordinate. We align a fit to a reference by minimizing total squared distance between their response profiles over permutations,

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_K} \sum_{k=1}^K \|\rho_{\cdot, \sigma(k)}^{\text{fit}} - \rho_{\cdot, k}^{\text{ref}}\|^2, \quad (6)$$

solved exactly by the Hungarian algorithm (Kuhn 1955; Hornik 2005) rather than by heuristic matching. Alignment is applied to replicate refits before they enter the variance, to the unweighted benchmark before comparison, and to any comparison across K .

6 Model selection

An information criterion compares a maximized log-likelihood against a penalty in units of $\log(\text{sample size})$, and the two must sit on the same scale. Base weights rarely sum to n . Because ℓ_w is linear in the weights, multiplying it by $n / \sum_i w_i$ is exactly equivalent to refitting with weights rescaled to sum to n : point estimates are untouched and only the criteria change. With $\ell_w^* = \ell_w \cdot n / \sum_i w_i$,

$$\text{AIC} = -2\ell_w^* + 2d_K, \quad \text{BIC} = -2\ell_w^* + d_K \log n. \quad (7)$$

Without this rescaling, weights summing to substantially more than n inflate the log-likelihood relative to a fixed penalty and BIC leans toward too many segments (Lumley and Scott 2015).

We report alongside these the relative entropy of the posterior classification, on the weighted scale,

$$R_E^2 = 1 - \frac{-\sum_i w_i \sum_k p_{ik} \log p_{ik}}{(\sum_i w_i) \log K}, \quad (8)$$

which indexes classification sharpness (Ramaswamy et al. 1993). The numerator is the weighted average posterior uncertainty and the denominator its maximum, attained when every respondent's posterior is uniform over the K segments, so R_E^2 runs from 0 at no classification information to 1 at perfect separation; values near 0.8 and above are conventionally read as good classification. Weighting is not cosmetic here: an unweighted entropy would describe the achieved sample's classification sharpness while every other quantity in the analysis describes the population.

The bootstrap likelihood ratio test, often preferred for class enumeration under simple random sampling (Nylund, Asparouhov, and Muthén 2007), is omitted by design. Its parametric bootstrap draws new samples of independent observations from the fitted model, which is not the process that generated a stratified clustered sample. Mplus disables the test under complex-design analysis for the same reason, and nothing is gained by reporting a p-value whose null distribution is known to be wrong.

The number of segments is never selected automatically. The analyst reviews the criteria, the entropy, the profiles, and the diagnostics of Section 9, then fixes K in configuration, where the decision is recorded and auditable.

```
enum <- future_map(cfg$K_range, function(K) {
  fit <- fit_lca(dat, w_vec, cats, cfg$items, K, start_seeds(cfg, K))
  ll_n <- fit$ll * scale_ic
  d <- df_k(K, cats)
  tibble(K = K, logLik = ll_n, df = d,
    AIC = -2 * ll_n + 2 * d,
    BIC = -2 * ll_n + d * log(n_obs),
    entropy = entropy_R2(fit$post, w_vec, K),
    converged = fit$converged)
}, .options = furrr_options(seed = NULL,
  packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr"))) |>
list_rbind()

enum |>
  mutate(across(c(logLik, AIC, BIC), \(x) round(x, 1)),
    entropy = round(entropy, 3)) |>
  lca_table(align = "r",
    caption = "Weighted fit statistics by number of segments. The log-likelihood is on
```

Table 3: Weighted fit statistics by number of segments. The log-likelihood is on the sum-to-n scale of Equation 6. Nothing is selected here.

K	logLik	df	AIC	BIC	entropy	converged
2	-32115.8	157	64545.7	65375.0	0.903	TRUE
3	-30686.4	236	61844.8	63091.4	0.913	TRUE

Table 3: Weighted fit statistics by number of segments. The log-likelihood is on the sum-to-n scale of Equation 6. Nothing is selected here. (*continued*)

K	logLik	df	AIC	BIC	entropy	converged
4	-29945.9	315	60521.9	62185.7	0.921	TRUE
5	-29526.9	394	59841.8	61922.9	0.908	TRUE
6	-29223.8	473	59393.6	61892.0	0.895	TRUE
7	-29004.9	552	59113.7	62029.4	0.893	TRUE
8	-28855.3	631	58972.7	62305.6	0.890	TRUE
9	-28707.5	710	58835.1	62585.3	0.887	TRUE
10	-28588.4	789	58754.9	62922.4	0.896	TRUE

```
enum |>
  select(K, BIC, entropy) |>
  pivot_longer(-K, names_to = "criterion", values_to = "value") |>
  mutate(criterion = recode(criterion, BIC = "Weighted BIC",
                           entropy = "Relative entropy")) |>
  ggplot(aes(K, value)) +
  geom_line(linewidth = 0.6, color = viridisLite::viridis(1, begin = 0.4)) +
  geom_point(size = 2.2, color = viridisLite::viridis(1, begin = 0.4)) +
  facet_wrap(~ criterion, scales = "free_y") +
  scale_x_continuous(breaks = cfg$K_range) +
  labs(x = "Number of segments (K)", y = NULL) +
  theme_lca()
```

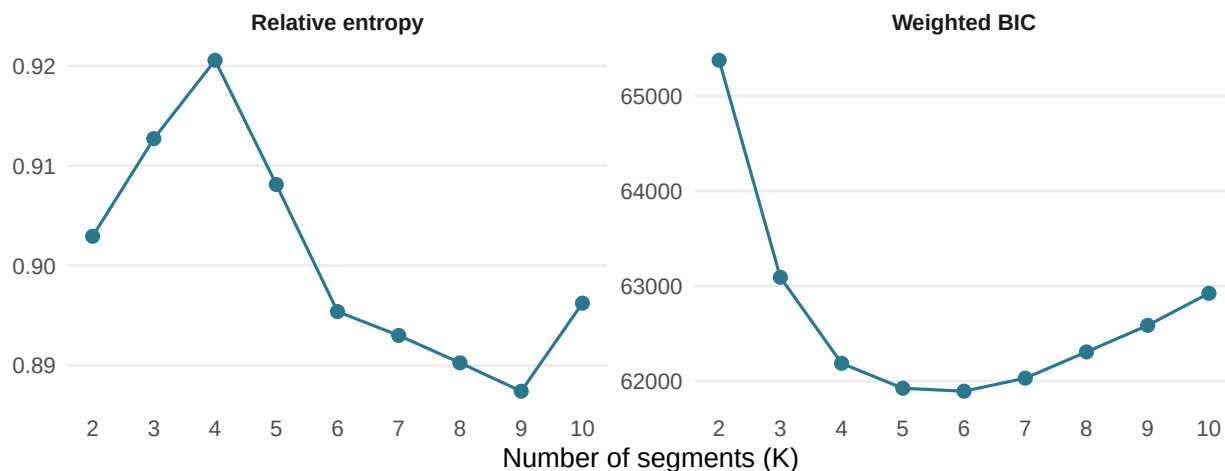


Figure 2: Enumeration diagnostics across K. BIC penalizes model dimension; relative entropy indexes classification sharpness. Neither decides alone.

7 The chosen model

```

fit_w <- fit_lca(dat, w_vec, cats, cfg$items, K_star,
               start_seeds(cfg, K_star), maxit = 5000L, tol = 1e-10)
fit_u <- align_to(fit_lca(dat, rep(1, n_obs), cats, cfg$items, K_star,
                          start_seeds(cfg, K_star), maxit = 5000L, tol = 1e-10),
                 fit_w)

inp <- make_inputs(dat, cfg$items, cats)
post_w <- posterior_of(fit_w$pi, fit_w$rho, inp$Y)
colnames(post_w) <- paste0("post_segment", seq_len(K_star))
modal_int <- max.col(post_w, ties.method = "first")

cat(sprintf(paste0("K = %d on %d cases from %d starts. Converged: %s.\n",
                  "Weighted log-likelihood (sum-to-n scale): %.1f\n",
                  "Weighted relative entropy: %.3f | mean maximum posterior: %.3f\n"),
            K_star, n_obs, cfg$n_starts,
            if (isTRUE(fit_w$converged)) "yes" else "NO, inspect before trusting",
            fit_w$ll * scale_ic, entropy_R2(post_w, w_vec, K_star),
            mean(post_w[cbind(seq_len(n_obs), modal_int)])))

```

```

K = 5 on 1454 cases from 200 starts. Converged: yes.
Weighted log-likelihood (sum-to-n scale): -29526.9
Weighted relative entropy: 0.908 | mean maximum posterior: 0.940

```

8 Design-based variance estimation

All uncertainty in this document comes from stratified delete-one-PSU jackknife replication (Wolter 2007; Rust and Rao 1996; Lumley 2004). With a_h primary sampling units (PSUs, or clusters) in stratum h , the replicate that deletes PSU α of stratum h assigns weight zero to that PSU's respondents, multiplies the weights of the remaining respondents in stratum h by $a_h/(a_h - 1)$, and leaves other strata unchanged. Writing $\hat{\theta}_{(h\alpha)}$ for the estimator recomputed on those weights,

$$\widehat{\mathbf{V}}(\hat{\theta}) = \sum_{h=1}^H \frac{a_h - 1}{a_h} \sum_{\alpha=1}^{a_h} (\hat{\theta}_{(h\alpha)} - \hat{\theta})(\hat{\theta}_{(h\alpha)} - \hat{\theta})^\top, \quad \nu = \sum_h a_h - H, \quad (9)$$

where ν is the design degrees of freedom, PSUs minus strata, used for every t reference in this document rather than a normal quantile.

Replication is a choice here, not a necessity: a pseudo-likelihood does admit a sandwich variance (Binder 1983), and that is what Mplus and Stata compute under survey estimation. We choose replication for three reasons specific to this problem. The parameter vector is large, 394 free parameters in this application, with many parameters near the boundary, and the linearized sandwich requires inverting an information matrix of that dimension. Replication propagates to new statistics without rederivation: once the replicate weights exist, the domain estimates of Section 14 inherit design-based variance by recomputation, where linearization would need a fresh delta-method derivation

at each stage. And replication makes the design specification auditable, since a misspecified cluster identifier changes the replicate structure visibly.

Each replicate refits the weighted EM on the replicate weights, warm-started from the full-sample solution and aligned to it by Equation 6. The resulting variance is therefore **within-mode** by construction: it expresses design variance around the reported solution, not uncertainty about which mode the likelihood surface should have selected.

A stratum containing a single PSU cannot receive the $a_h/(a_h-1)$ scaling. The `survey.lonely.psu` option governs linearization variance and does not change how replicate weights are constructed, so a singleton stratum would contribute no variance and the resulting standard errors would be anticonservative in exactly the strata where the data are thinnest. The workflow therefore treats singleton strata as a hard stop, and performs the check on the analysis frame rather than the released file, because case exclusions can create singletons the provider's data does not have.

Intervals for every probability are formed on the logit scale by the delta method. Since $d \logit(p)/dp = 1/\{p(1-p)\}$,

$$se_{\logit}(\hat{p}) = \frac{se(\hat{p})}{\hat{p}(1-\hat{p})}, \quad CI = \logit^{-1}(\logit(\hat{p}) \pm t_{0.975, \nu} se_{\logit}(\hat{p})), \quad (10)$$

which keeps intervals inside $(0, 1)$ without truncation and improves coverage near the boundary. Estimates within 10^{-3} of a boundary have no meaningful logit-scale standard error and receive Wald intervals truncated to $[0, 1]$, flagged in the delivered tables.

```
design <- build_rep_design(dat, cfg)
des <- design$des
rep_des <- design$rep_des
nu <- degf(rep_des)
crit <- qt(0.975, df = nu)

cat(sprintf("Design: %d replicates, %d degrees of freedom.\n",
           ncol(rep_des$repweights), nu))
```

Design: 130 replicates, 126 degrees of freedom.

```
# Parameter bookkeeping, in the order flatten_fit() produces.
param_meta <- bind_rows(
  tibble(kind = "pi", item = NA_character_, category = NA_integer_,
         segment = seq_len(K_star)),
  imap(cats, function(Cj, item_name)
    expand_grid(segment = seq_len(K_star), category = seq_len(Cj)) |>
    transmute(kind = "rho", item = item_name, category, segment)) |>
  list_rbind())

flatten_fit <- function(fit) c(fit$pi, unlist(map(fit$rho, as.vector)))

theta_fun <- function(w_rep) {
```

```

fit <- em_run(inp$Y, inp$OH, cats, w_rep, K_star,
             init = fit_w[c("pi", "rho")], maxit = 1000L, tol = 1e-10)
flatten_fit(align_to(fit, fit_w))
}

est_vec <- flatten_fit(fit_w)
se_vec <- sqrt(diag(replicate_variance(rep_des, theta_fun, est_vec)))

ci_tbl <- param_meta |>
  mutate(estimate = as.numeric(est_vec), se = as.numeric(se_vec),
         boundary = estimate < 1e-3 | estimate > 1 - 1e-3,
         p_c = pmin(pmax(estimate, 1e-6), 1 - 1e-6),
         gse = se / (p_c * (1 - p_c)),
         lo = if_else(boundary, pmax(0, estimate - crit * se),
                      plogis(qlogis(p_c) - crit * gse)),
         hi = if_else(boundary, pmin(1, estimate + crit * se),
                      plogis(qlogis(p_c) + crit * gse))) |>
  select(-p_c, -gse)

```

9 Fit diagnostics

The chosen model is diagnosed before it is named or interpreted. Three questions are asked in order: does each item separate the segments, does each item earn the parameters it costs, and does the local independence assumption of Equation 1 hold pairwise. A model that fails these is refit at a different K or with items removed, and only a model that passes proceeds to Section 11.

Item discrimination summarizes how much item j separates the segments, as the mean over segment pairs of the total variation distance between their conditional response distributions:

$$D_j = \frac{2}{K(K-1)} \sum_{k < l} \frac{1}{2} \sum_{c=1}^{C_j} |\rho_{jck} - \rho_{jcl}|. \quad (11)$$

The inner expression is total variation distance between two discrete distributions, bounded in $[0, 1]$ and equal to the largest difference in probability the two assign to any event; the leading factor is the reciprocal of the number of pairs, $K(K-1)/2$, so D_j is a mean of quantities each in $[0, 1]$ and is itself in $[0, 1]$. A value of zero means every segment answers item j identically, making it a candidate for removal in a sensitivity refit; values near one mean the item alone nearly determines segment membership.

```

item_discrim <- item_discrimination(fit_w, cfg$items)

item_discrim |>
  mutate(discrimination = round(discrimination, 3)) |>
  lca_table(col.names = c("Item", "Discrimination"), align = "lr",

```

```
widths = c("16em", "10em"),
caption = "Item discrimination from Equation 13: mean over segment pairs of the to
```

Table 4: Item discrimination from Equation 13: mean over segment pairs of the total variation distance between their response distributions. Bounded in $[0, 1]$; zero means every segment answers alike.

Item	Discrimination
justice_system	0.757
public_ministry	0.706
auditor	0.699
supreme_court	0.689
legislature	0.664
elections	0.658
political_parties	0.631
electoral_tribunal	0.621
municipality	0.620
police	0.585
media	0.490
armed_forces	0.485
president	0.457

Discrimination is an effect size; a companion screen asks whether each item earns the $K(C_j - 1)$ parameters it costs. For item j , compare the fitted model against one in which item j is segment-independent. Under local independence that restricted model factorizes into a latent class model on the remaining items multiplied by item j 's weighted marginal, so its pseudo-log-likelihood is available without a constrained refit and the two models' criteria are directly comparable:

$$\Delta\text{BIC}_j = \text{BIC}(\text{item } j \text{ segment-independent}) - \text{BIC}(\text{full model}). \quad (12)$$

A negative value means BIC prefers dropping item j 's segment dependence. This is a ranking under a pseudo-likelihood and not a Bayes factor, so the conventional interpretive thresholds for BIC differences do not apply and are not quoted.

```
ll_full <- fit_w$ll * scale_ic
df_full <- df_k(K_star, cats)

marginal_ll <- function(item) {
  y <- as.integer(dat[[item]])
  p <- as.numeric(xtabs(w_vec ~ y))
  sum(w_vec * log((p / sum(p))[y]))
}

item_screen <- item_discrim |>
  mutate(delta_bic = future_map_dbl(item, function(j) {
    keep_items <- setdiff(cfg$items, j)
    fit_wo <- fit_lca(dat, w_vec, cats[keep_items], keep_items, K_star,
                      start_seeds(cfg, K_star))
    ll_restr <- (fit_wo$ll + marginal_ll(j)) * scale_ic
```

```

df_restr <- df_k(K_star, cats[keep_items]) + (cats[[j]] - 1)
(-2 * ll_restr + df_restr * log(n_obs)) - (-2 * ll_full + df_full * log(n_obs))
}, .options = furrr_options(seed = NULL,
  packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr")))) |>
arrange(delta_bic)

item_screen |>
mutate(discrimination = round(discrimination, 3), delta_bic = round(delta_bic, 1)) |>
lca_table(align = "lrr", col.names = c("Item", "Discrimination", "Delta BIC"),
  widths = c("14em", "10em", "8em"),
  caption = "Indicator screen from Equation 14, most droppable first. Negative flags

```

Table 5: Indicator screen from Equation 14, most droppable first. Negative flags an item whose segment dependence BIC does not support.

Item	Discrimination	Delta BIC
president	0.457	336.9
armed_forces	0.485	437.6
media	0.490	501.3
police	0.585	755.6
electoral_tribunal	0.621	860.8
political_parties	0.631	895.8
municipality	0.620	908.3
elections	0.658	1014.9
legislature	0.664	1016.5
supreme_court	0.689	1154.2
public_ministry	0.706	1190.9
auditor	0.699	1227.7
justice_system	0.757	1460.1

Local independence is probed pairwise. For items a and b , let $\widehat{P}_{ab}(c, d)$ be the design-weighted observed proportion giving response c to item a and d to item b , and let

$$\Pi_{ab} = \rho_a \text{diag}(\pi) \rho_b^\top \quad (13)$$

be the model-implied joint distribution, where ρ_a is the $C_a \times K$ matrix of conditional probabilities for item a . Equation 13 is the model's own statement about the pair: condition on segment, multiply the two conditional distributions because the model asserts they are independent given segment, and average over segments weighted by prevalence. The bivariate residual is the total variation distance between observed and implied,

$$\text{BVR}_{ab} = \frac{1}{2} \sum_{c,d} |\widehat{P}_{ab}(c, d) - \Pi_{ab}(c, d)| \in [0, 1]. \quad (14)$$

This departs from the Pearson-type statistic conventionally reported, and deliberately. Oberski, Kollenburg, and Vermunt (2013) showed that the Pearson statistic's null distribution is already not chi-square for unweighted binary data, recommending bootstrap calibration; under design weighting and a pseudo-likelihood on a clustered sample no reference distribution is valid at all. A Rao-Scott

design adjustment would repair the design component but not the fact that the residual's null is not chi-square even under simple random sampling, so it does not rescue the statistic here. A statistic without a null distribution is an effect size, and an effect size should sit on an interpretable scale: Equation 14 is bounded in $[0, 1]$, is exactly zero under perfect local independence, reads as the largest probability discrepancy over any event in the two-way table, and shares its scale with Equation 11. It also removes a numerical failure of the Pearson form, which divides by the model-implied cell probability and so can be dominated by a single near-empty corner of a large table.

```
bvr_tbl <- bvr_pairs(dat, w_vec, cfg$items, fit_w)

bvr_tbl |>
  mutate(item_a = factor(item_a, levels = cfg$items),
         item_b = factor(item_b, levels = cfg$items)) |>
  ggplot(aes(item_a, item_b, fill = bvr)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", bvr)), size = 2.0) +
  scale_fill_viridis_c(name = "BVR", direction = -1) +
  labs(x = NULL, y = NULL) +
  theme_lca() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

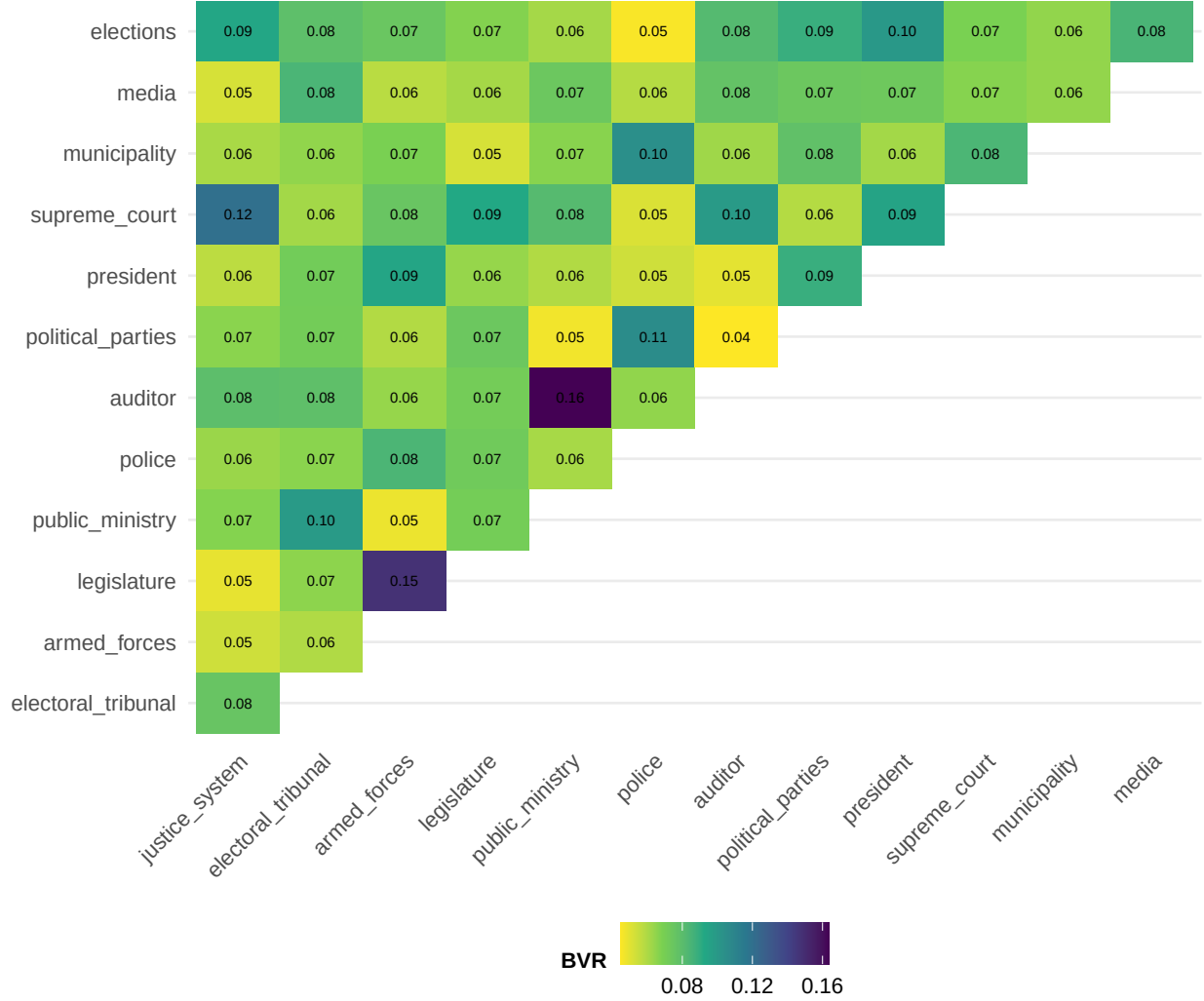


Figure 3: Bivariate residuals from Equation 16 as total variation distance. Darker cells are item pairs whose association the segments explain least well. Persistent dark pairs argue for dropping or merging an item, or trying $K+1$.

10 Validation against an independent implementation

At unit weights, Equation 3 reduces to the ordinary log-likelihood, so the weighted EM and **poLCA** (Linzer and Lewis 2011) maximize the same function and must agree up to optimizer tolerance and mode selection. This is enforced as a runtime check rather than asserted.

The comparison fits a third candidate by warm-starting our own EM at the **poLCA** solution, so whichever basin attains the higher log-likelihood is retained and the comparison is between optima rather than between stopping rules. Where a gap appears, the two log-likelihoods separate the possible causes: equal log-likelihoods with different parameters indicate drift along a flat ridge under the stopping rule, a higher log-likelihood here indicates an inferior **poLCA** mode, and a higher log-likelihood there indicates our own search needs more starts.

This check validates estimation. It says nothing about the variance implementation of Section 8, which is a separate concern and a reason to inspect the replicate diagnostics rather than treat agreement here as global.

```
polca_df <- dat |> select(all_of(cfg$items)) |> as.data.frame()
polca_f <- as.formula(paste0("cbind(", paste(cfg$items, collapse = ", "), ") ~ 1"))

set.seed(cfg$seed + 7L)
pl <- poLCA(polca_f, data = polca_df, nclass = K_star, nrep = 10,
            maxiter = 5000, na.rm = TRUE, verbose = FALSE)
pl_aligned <- align_to(list(pi = as.numeric(pl$P), rho = map(pl$probs, t),
                           post = pl$posterior), fit_w)

fit_from_pl <- em_run(inp$Y, inp$OH, cats, rep(1, n_obs), K_star,
                    init = list(pi = pl_aligned$pi, rho = pl_aligned$rho),
                    maxit = 5000L, tol = 1e-10)
fit_unit <- if (fit_from_pl$ll > fit_u$ll) align_to(fit_from_pl, fit_w) else fit_u

validation <- tibble(
  max_pi_gap = max(abs(fit_unit$pi - pl_aligned$pi)),
  max_rho_gap = max(map2_dbl(fit_unit$rho, pl_aligned$rho, \(a, b) max(abs(a - b))))
validation$match <- max(validation$max_pi_gap, validation$max_rho_gap) < 0.005

cat(sprintf(paste0("VALIDATION (unit weights, our EM vs poLCA):\n",
  " max |prevalence gap| = %.5f\n",
  " max |conditional-probability gap| = %.5f\n",
  " log-likelihood ours = %.4f | poLCA = %.4f\n -> %s\n"),
  validation$max_pi_gap, validation$max_rho_gap, fit_unit$ll, pl$llik,
  if (validation$match) "MATCH: implementations agree."
  else "MISMATCH: investigate before trusting the weighted fit."))
```

```
VALIDATION (unit weights, our EM vs poLCA):
max |prevalence gap| = 0.00000
max |conditional-probability gap| = 0.00000
log-likelihood ours = -29526.8844 | poLCA = -29526.8844
-> MATCH: implementations agree.
```

11 Naming the segments

Segment naming is drafted by a language model under a fixed instrument and is a convenience with an audited override path, not a measurement claim. Recent evidence suggests language models perform annotation-adjacent tasks reliably when tightly instructed (Gilardi, Alizadeh, and Kubli 2023; Ziemis et al. 2024), which is the narrow basis on which the step is used here. In TSE terms it is a processing step (Section 2): it occurs after every number is computed, produces a display string, and never enters a computation, so a wrong name is a presentation error rather than a statistical one.

The instrument comprises a persona and rule set directing interpretation strictly from the displayed conditional response probabilities and item wording, one isolated call per segment, an optional survey-context paragraph fenced to referent resolution only, and a collision-gated harmonization pass that may edit labels but never descriptions. Isolation is empirical: joint prompts, in which the model sees all segments at once and must assign names one-to-one, confused near-neighbor segments and allowed a single confusion to corrupt two labels.

Reproducibility rests on a freeze file rather than a sampling seed. When `segment_labels.csv` exists in the output directory it is used and validated against K ; otherwise the model drafts once and writes it. Editing the file is how the analyst takes over naming, and deleting it triggers a redraft. A seed would make the draft reproducible only for a fixed model, endpoint, and package version; the file is reproducible unconditionally.

Labels are drafts. Naming happens only after the diagnostics of Section 9 and the validation of Section 10 have been read, and each label must be verified against its profile panel in Figure 4 before it is quoted.

```
cat("===== SYSTEM PROMPT =====\n")
```

```
===== SYSTEM PROMPT =====
```

```
cat(wrap_text(lca_persona()), "\n\n")
```

You are a senior survey methodologist who reads latent class analysis (LCA) measurement models. In this work each latent class is called a SEGMENT; that is a word-choice preference and the statistical object is unchanged. Each segment is described only by its item-response probabilities: for every survey item, the probability that a member of that segment gives each answer. A segment leans toward the answers with high probability. You interpret a segment strictly from these probabilities and the item wording, never from outside assumptions.

```
cat("===== USER PROMPT (segment 1) =====\n")
```

```
===== USER PROMPT (segment 1) =====
```

```
cat(wrap_text(prompt_segment_label(fit_w, 1, dictionary, cfg$items,
                                cfg$survey_context)), "\n")
```

SURVEY CONTEXT

These items come from the 2023 AmericasBarometer survey of Mexico, conducted by the LAPOP Lab at Vanderbilt University. The AmericasBarometer is a comparative public opinion study of democratic attitudes and governance across the Americas, fielded to national probability samples of voting-age adults.

The battery analyzed here measures trust in national institutions: respondents rate,

for each institution, how much they trust it on a seven-point scale anchored at 1 (not at all) and 7 (a lot). The segments summarize patterns of institutional trust across these items.

ONE SEGMENT FROM A LATENT CLASS ANALYSIS (LCA) MEASUREMENT MODEL

SEGMENT 1 (estimated prevalence 34%):

justice_system "Confianza en el sistema de justicia"

P(Nada)=0.00, P(2)=0.04, P(3)=0.16, P(4)=0.45, P(5)=0.28, P(6)=0.06, P(Mucho)=0.01

electoral_tribunal "Confianza en el Tribunal Supremo Electoral"

P(Nada)=0.03, P(2)=0.06, P(3)=0.12, P(4)=0.27, P(5)=0.28, P(6)=0.17, P(Mucho)=0.08

armed_forces "Confianza en las fuerzas armadas"

P(Nada)=0.02, P(2)=0.03, P(3)=0.07, P(4)=0.17, P(5)=0.27, P(6)=0.25, P(Mucho)=0.20

legislature "Confianza en la legislatura"

P(Nada)=0.02, P(2)=0.03, P(3)=0.11, P(4)=0.38, P(5)=0.30, P(6)=0.13, P(Mucho)=0.04

public_ministry "Confianza en el Ministerio Público"

P(Nada)=0.01, P(2)=0.03, P(3)=0.10, P(4)=0.36, P(5)=0.35, P(6)=0.11, P(Mucho)=0.04

police "Confianza en la policía nacional"

P(Nada)=0.08, P(2)=0.18, P(3)=0.28, P(4)=0.26, P(5)=0.14, P(6)=0.05, P(Mucho)=0.02

auditor "Confianza en la Auditoría Superior de la Federación"

P(Nada)=0.01, P(2)=0.02, P(3)=0.14, P(4)=0.41, P(5)=0.30, P(6)=0.10, P(Mucho)=0.02

political_parties "Confianza en los partidos políticos"

P(Nada)=0.11, P(2)=0.19, P(3)=0.27, P(4)=0.29, P(5)=0.11, P(6)=0.03, P(Mucho)=0.01

president "Confianza en el presidente"

P(Nada)=0.04, P(2)=0.06, P(3)=0.07, P(4)=0.17, P(5)=0.19, P(6)=0.21, P(Mucho)=0.27

supreme_court "Confianza en el tribunal supremo"

P(Nada)=0.01, P(2)=0.04, P(3)=0.12, P(4)=0.31, P(5)=0.36, P(6)=0.14, P(Mucho)=0.04

municipality "Confianza en su municipalidad"

P(Nada)=0.01, P(2)=0.04, P(3)=0.14, P(4)=0.36, P(5)=0.33, P(6)=0.09, P(Mucho)=0.02

media "Confianza en los medios de comunicación"

P(Nada)=0.05, P(2)=0.11, P(3)=0.21, P(4)=0.26, P(5)=0.25, P(6)=0.08, P(Mucho)=0.04

elections "Confianza en las elecciones"

P(Nada)=0.02, P(2)=0.05, P(3)=0.21, P(4)=0.30, P(5)=0.26, P(6)=0.13, P(Mucho)=0.03

TASK

Read this single segment and return: a short DRAFT label (2 to 5 words) for an analyst to refine, and a one or two sentence factual description anchored to its high-probability answers.

RULES:

1. Use only the response probabilities and item wording shown. Survey context only clarifies what the items refer to; attribute nothing to the segment that the probabilities do not show.
 2. Anchor every statement to the high-probability answers of this segment.
 3. If the profile is diffuse (no clear high-probability answers), say so.
 4. Return only valid JSON: no prose before or after, no markdown fences.
- JSON (one object): {"label": "...", "description": "..."}

```
segment_labels <- get_segment_labels(fit_w, dictionary, cfg$items, cfg)

lca_table(segment_labels, widths = c("2em", "11em", "28em"),
  caption = "Segment labels and descriptions in use. Drafts, to verify against the response profiles below.")
```

Table 6: Segment labels and descriptions in use. Drafts, to verify against the response profiles below.

K	Label	Description
1	Guarded System Trust	This segment generally expresses moderate trust in most national institutions, with response probabilities peaking at a '4' (some trust) for many items like the justice system, auditor, and supreme court. However, they exhibit notably lower trust in political parties and the police, with higher probabilities of expressing 'no trust at all' or 'little trust' for these institutions.
2	Reserved Institutional Trust	This segment generally expresses moderate trust in most national institutions, frequently selecting a '5' or '6' on the seven-point scale. While outright distrust ('Nada') is rare, strong trust ('Mucho') is also uncommon, suggesting a cautious or reserved level of confidence.
3	Widespread Distrust	This segment expresses very low trust across nearly all national institutions, with probabilities exceeding 0.60 for 'not at all' on most items. While showing slightly more trust in the president and municipality than other institutions, even these remain relatively low.
4	Cautious, Low Trust	This segment generally expresses moderate levels of trust in most national institutions, frequently selecting a rating of '3' on the seven-point scale. However, they exhibit notably lower trust in political parties and the police, with a high probability of responding 'not at all' to questions about these institutions.
5	Presidential & Judicial Trust	This segment expresses strong trust in the president (82% probability of 'A lot') and generally high trust in the justice system, electoral tribunal, armed forces, public ministry, and supreme court (all above 70% probability of 'A lot'). Trust in other institutions is more moderate, with notably lower trust in political parties, the police, and the media.

```
lab_lev <- str_wrap(paste0("S", segment_labels$K, ": ", segment_labels$Label), 22)
lab_of <- function(k) factor(lab_lev[k], levels = lab_lev)
lab_line <- function(k) paste0("S", k, ": ", segment_labels$Label[k])
lab_short <- function(k) paste0("S", k)
```

12 The measurement model

```
pi_ci <- ci_tbl |>
  filter(kind == "pi") |>
  transmute(K = segment, est = estimate, lo, hi)

pi_ci |>
```

```
mutate(Segment = lab_line(K),
       `Weighted prevalence [95% CI]` = sprintf("%.3f [%.3f, %.3f]", est, lo, hi),
       `Unweighted` = sprintf("%.3f", fit_u$pi[K]),
       Gap = sprintf("%+.3f", est - fit_u$pi[K])) |>
select(Segment:Gap) |>
lca_table(align = "lrrr", widths = c("13em", "13em", "8em", "5em"),
          caption = "Segment prevalences: design-weighted estimates with 95 percent JKn inter
```

Table 7: Segment prevalences: design-weighted estimates with 95 percent JKn intervals from Equations 11 and 12, against the unweighted fit. Read the gap against the unequal weighting effect reported in Section 3; when that is near 1 the gap is noise rather than correction.

Segment	Weighted prevalence [95% CI]	Unweighted	Gap
S1: Guarded System Trust	0.341 [0.300, 0.384]	0.341	+0.000
S2: Reserved Institutional Trust	0.207 [0.171, 0.249]	0.207	+0.000
S3: Widespread Distrust	0.107 [0.084, 0.135]	0.107	+0.000
S4: Cautious, Low Trust	0.234 [0.192, 0.281]	0.234	+0.000
S5: Presidential & Judicial Trust	0.112 [0.093, 0.134]	0.112	+0.000

```
ci_tbl |>
  filter(kind == "rho") |>
  mutate(Segment = lab_of(segment), item = factor(item, levels = cfg$items)) |>
  ggplot(aes(factor(category), item, fill = estimate)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", estimate)), size = 2.1) +
  scale_fill_viridis_c(name = "P", limits = c(0, 1)) +
  facet_wrap(~ Segment, ncol = min(3, K_star)) +
  labs(x = "Response category", y = NULL) +
  theme_lca() +
  theme(axis.text.y = element_text(size = 7))
```

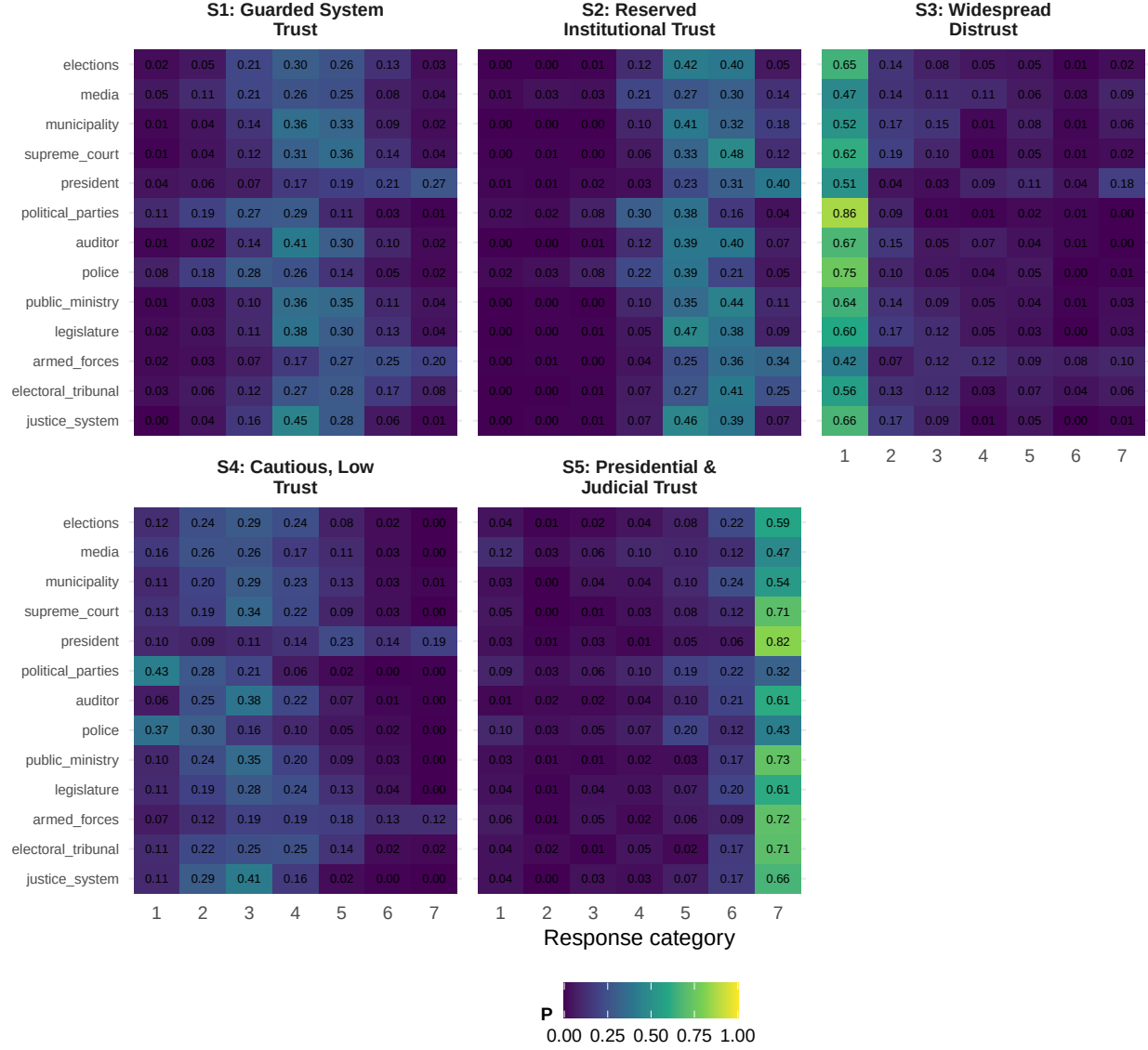


Figure 4: Conditional response probabilities $P(\text{response} | \text{segment})$ for every item, the parameters estimated by Equation 5. Rows are items, columns are response categories.

13 Scoring segment membership

Local independence makes assignment exact rather than approximate for item-partial respondents. A missing item drops out of the within-segment product, so for respondent i with answered item set A_i ,

$$p_{ik}^{\text{obs}} = \frac{\pi_k \prod_{j \in A_i} \rho_{j y_{ij} k}}{\sum_l \pi_l \prod_{j \in A_i} \rho_{j y_{ij} l}}, \quad \hat{X}_i = \arg \max_k p_{ik}^{\text{obs}}, \quad (15)$$

which is the exact posterior given what respondent i actually answered, under the model and under ignorability of the missingness pattern given segment. An information floor `cfg$min_items` is enforced: respondents answering fewer items receive no assignment rather than a weakly informed one.

Scoring precedes domain estimation, and all domain estimation runs on the scored frame rather than the estimation sample. This follows the logic of three-step latent class analysis, in which the assignment step covers every respondent it validly can, and it matters substantively rather than cosmetically: item nonresponse on an attitude battery correlates with education, age, and other characteristics of interest, so restricting domain estimation to complete responders introduces a selection into exactly the composition being estimated. This is the item-level nonresponse concern of Section 2.

The cost, and it is the correlated-error case noted there, is that respondents scored from fewer items carry flatter posteriors and therefore more classification error than those scored from all items. The replicate design is rebuilt on the scored frame and the singleton stratum check is repeated there.

```
pred <- predict_segments(survey_dat_full, fit_w, cfg$items, cfg$min_items)
survey_scored <- bind_cols(survey_dat_full, pred)

scored <- survey_scored |>
  filter(!is.na(segment)) |>
  mutate(segment = factor(segment, levels = seq_len(K_star)))

design_s <- build_rep_design(scored, cfg)
des_s <- design_s$des
rep_des_s <- design_s$rep_des
nu_s <- degf(rep_des_s)
crit_s <- qt(0.975, df = nu_s)

cat(sprintf(paste0("Estimation sample: %d item-complete cases.\n",
  "Scored sample: %d of %d respondents (floor: %d items), a gain of %d.\n",
  "Domain design: %d replicates, %d degrees of freedom.\n"),
  n_obs, nrow(scored), nrow(survey_scored), cfg$min_items,
  nrow(scored) - n_obs, ncol(rep_des_s$repweights), nu_s))
```

Estimation sample: 1454 item-complete cases.

Scored sample: 1618 of 1622 respondents (floor: 7 items), a gain of 164.

Domain design: 130 replicates, 126 degrees of freedom.

```
scored |>
  group_by(n_items_answered) |>
  summarise(n = n(), mean_max_posterior = round(mean(max_posterior), 3),
    .groups = "drop") |>
  lca_table(aligned = "rrr", widths = c("10em", "6em", "12em"),
    col.names = c("Items answered", "n", "Mean max posterior"),
    caption = "Assignment quality by number of items answered. Respondents scored from
```

Table 8: Assignment quality by number of items answered. Respondents scored from fewer items carry flatter posteriors and so more classification error. If the mean falls sharply at the low end, raise `cfg$min_items`.

Items answered	n	Mean max posterior
7	3	0.718
8	5	0.889
9	5	0.891
10	22	0.818
11	25	0.901
12	104	0.903
13	1454	0.940

14 Domain estimation

The quantity of interest is segment prevalence within demographic domains. Writing d for a level of a demographic variable,

$$\hat{p}_{dk} = \frac{\sum_{i \in d} w_i \mathbf{1}(\hat{X}_i = k)}{\sum_{i \in d} w_i}, \quad (16)$$

the design-weighted proportion of domain d assigned to segment k , with variance from the same JK_n replicates as Equation 9. In implementation this is `svyby` over the demographic with `svymean` of the segment factor inside each level, so the estimate and its variance come from the replicate design directly. Rows sum to one across segments by construction, which means the cells within a row are not independent: if one segment’s share rises, others must fall.

This estimator uses hard modal assignment and is therefore **attenuated**. Classification error pulls composition toward the marginal distribution, so differences between demographic levels are understated (Bolck, Croon, and Hagenaars 2004; Vermunt 2010). The direction is what makes this tractable: a difference that survives modal assignment is real, while a null under modal assignment may be a genuine difference that classification error erased. Bias-adjusted three-step corrections exist for this (Vermunt 2010; Bakk, Tekle, and Vermunt 2013; Bakk, Oberski, and Vermunt 2014); we computed the standard correction and it moved every estimate by less than one percentage point against standard errors several times larger, consistent with the high relative entropy reported in Section 6, so the uncorrected estimates are reported here and the attenuation is treated as a stated limitation rather than a modeled quantity.

```
marginals <- map(cfg$aux, function(v) {
  m <- svymean(reformulate(v), rep_des_s, na.rm = TRUE)
  raw <- scored |>
    filter(!is.na(.data[[v]])) |>
    count(.data[[v]], name = "n") |>
    set_names(c("level", "n"))
  tibble(variable = v,
    level = str_remove(names(coef(m)), paste0("^", v)),
    weighted = as.numeric(coef(m)),
    se = as.numeric(SE(m))) |>
```

```

    left_join(mutate(raw, level = as.character(level)), by = "level")
  }) |>
  list_rbind()

# svyby splits the design by the demographic and runs svymean(~segment) within
# each level. Columns are read by position (by-variable, then K estimates, then
# K SEs) because svyby's naming for a factor outcome varies by version.
domain <- map(cfg$aux, function(v) {
  sb <- as.data.frame(svyby(~ segment, reformulate(v), rep_des_s, svymean,
                           na.rm = TRUE))

  vals <- sb[, -1, drop = FALSE]
  stopifnot(ncol(vals) == 2 * K_star)
  tibble(variable = v,
          level = rep(as.character(sb[[1]]), times = K_star),
          segment = rep(seq_len(K_star), each = nrow(sb)),
          p = as.numeric(as.matrix(vals[, seq_len(K_star)])),
          se = as.numeric(as.matrix(vals[, K_star + seq_len(K_star)])))
}) |>
  list_rbind() |>
  filter(!is.na(level)) |>
  mutate(lo = pmax(0, p - crit_s * se), hi = pmin(1, p + crit_s * se))

```

```

walk(cfg$aux, function(v) {
  tag <- str_replace_all(v, "_", "-")
  cat("\n\n##", v, "\n\n")

  marginals |>
    filter(variable == v) |>
    transmute(Level = level, n, Share = sprintf("%.3f (%.3f)", weighted, se)) |>
    lca_table(align = "lrr", widths = c("12em", "4em", "12em"),
              label = paste0("comp-", tag),
              caption = paste0("Composition by ", v,
                                ": design-weighted proportion among respondents ",
                                "who answered the item, with its JKn standard error.")) |>

    cat(sep = "\n")
  cat("\n\n")

  domain |>
    filter(variable == v) |>
    transmute(Level = level, Segment = lab_short(segment),
              cell = sprintf("%.3f (%.3f)", p, se)) |>
    pivot_wider(names_from = Segment, values_from = cell) |>
    lca_table(align = str_c("l", strrep("r", K_star)),
              widths = c("12em", rep("6em", K_star)),
              label = paste0("dom-", tag),
              caption = paste0("Segment prevalence within each level of ", v,
                                " from Equation 18, with JKn standard errors. ",

```

```

                                "Rows sum to one across segments.)) |>
  cat(sep = "\n")
  cat("\n\n")

p <- domain |>
  filter(variable == v) |>
  mutate(Segment = lab_of(segment)) |>
  ggplot(aes(p, level, color = Segment)) +
  geom_pointrange(aes(xmin = lo, xmax = hi), size = 0.35, linewidth = 0.4,
                  position = position_dodge(width = 0.6)) +
  scale_color_viridis_d(begin = 0.1, end = 0.8) +
  scale_x_continuous(limits = c(0, NA)) +
  labs(x = "Prevalence of segment membership", y = NULL, color = NULL, title = v) +
  theme_lca()
print(p)
cat("\n\n{{< pagebreak >}}\n\n")
})

```

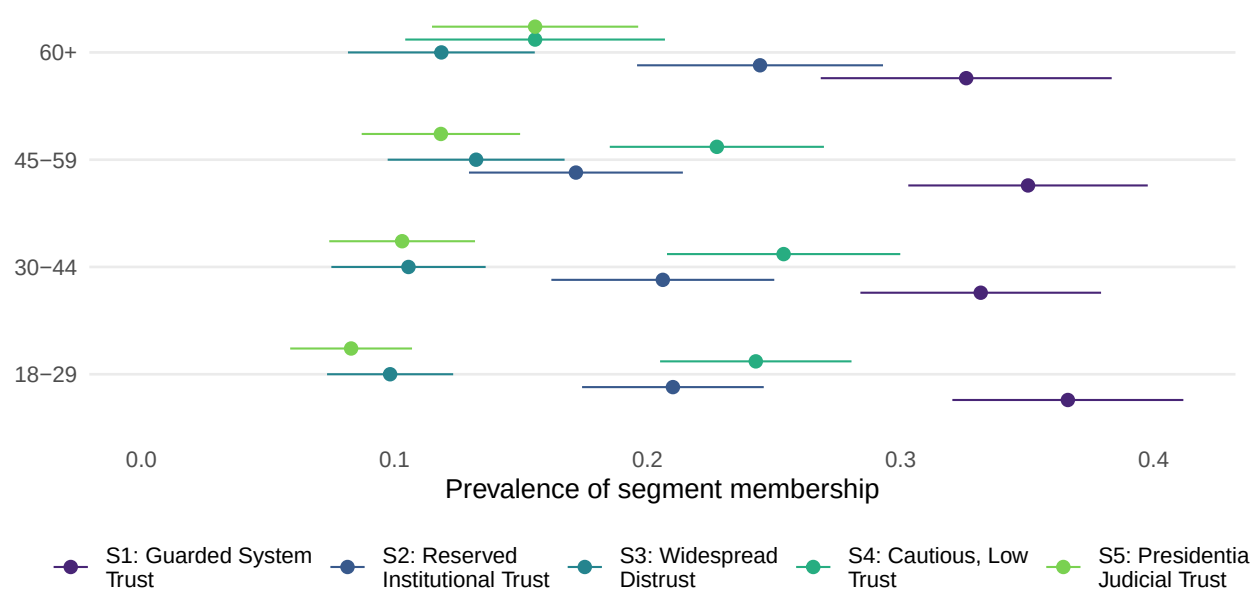
14.1 age_cat

Table 9: Composition by age_cat: design-weighted proportion among respondents who answered the item, with its JK_n standard error.

Level	n	Share
18-29	519	0.321 (0.005)
30-44	398	0.246 (0.009)
45-59	431	0.266 (0.011)
60+	270	0.167 (0.011)

Table 10: Segment prevalence within each level of age_cat from Equation 18, with JK_n standard errors. Rows sum to one across segments.

Level	S1	S2	S3	S4	S5
18-29	0.366 (0.023)	0.210 (0.018)	0.098 (0.013)	0.243 (0.019)	0.083 (0.012)
30-44	0.332 (0.024)	0.206 (0.022)	0.106 (0.015)	0.254 (0.023)	0.103 (0.015)
45-59	0.350 (0.024)	0.172 (0.021)	0.132 (0.018)	0.227 (0.021)	0.118 (0.016)
60+	0.326 (0.029)	0.244 (0.025)	0.119 (0.019)	0.156 (0.026)	0.156 (0.021)



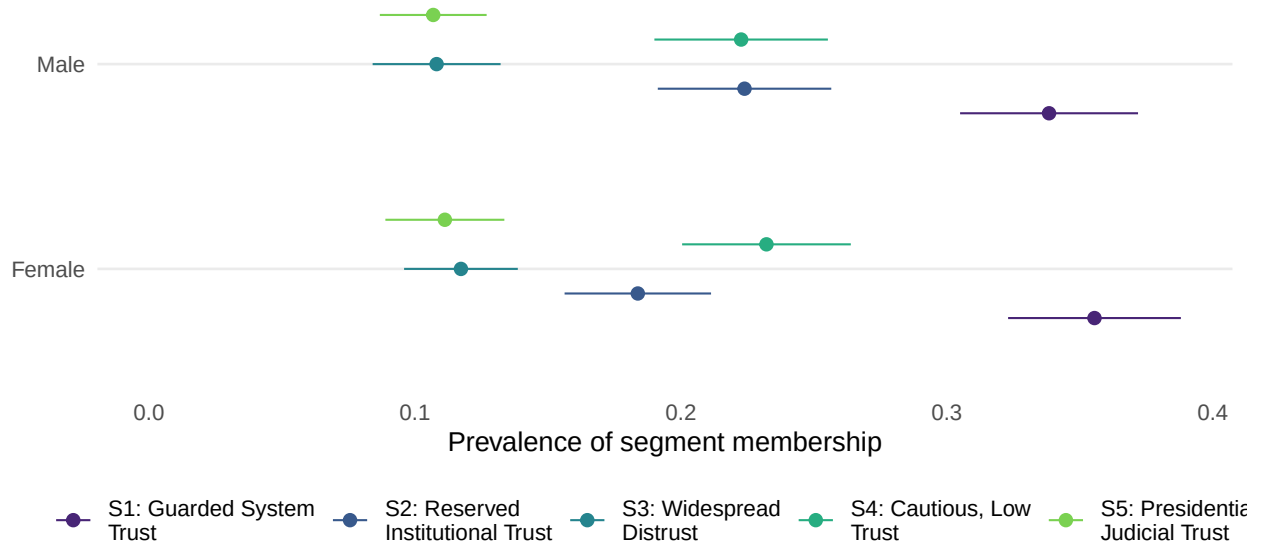
14.2 sex

Table 11: Composition by sex: design-weighted proportion among respondents who answered the item, with its JK_n standard error.

Level	n	Share
Male	786	0.487 (0.006)
Female	827	0.513 (0.006)

Table 12: Segment prevalence within each level of sex from Equation 18, with JK_n standard errors. Rows sum to one across segments.

Level	S1	S2	S3	S4	S5
Male	0.338 (0.017)	0.224 (0.016)	0.108 (0.012)	0.223 (0.016)	0.107 (0.010)
Female	0.356 (0.016)	0.184 (0.014)	0.117 (0.011)	0.232 (0.016)	0.111 (0.011)



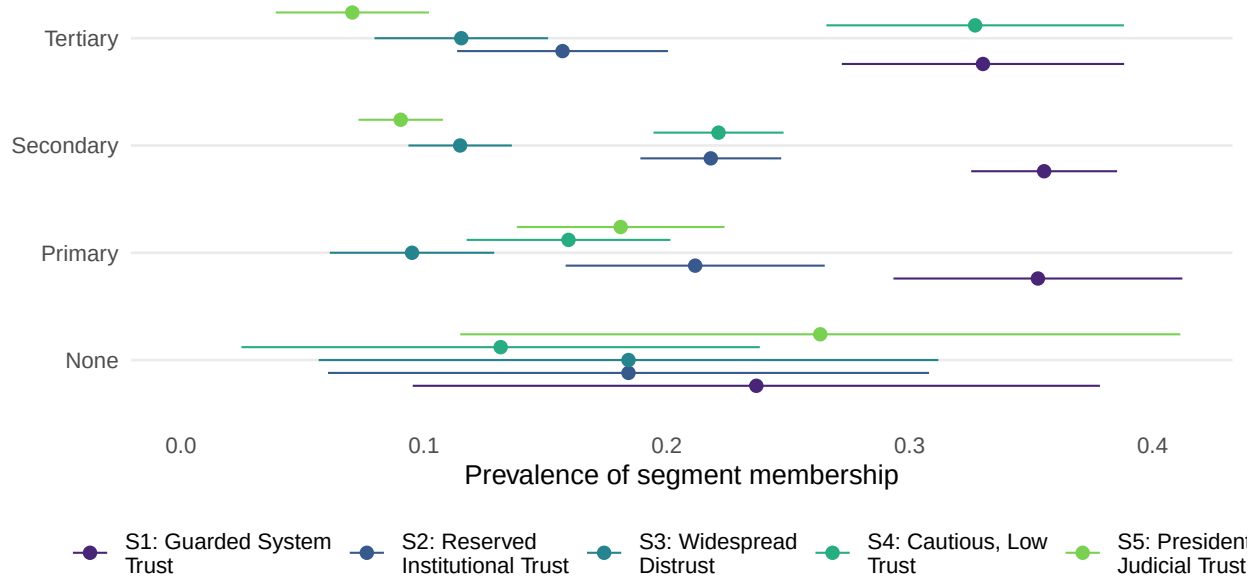
14.3 education

Table 13: Composition by education: design-weighted proportion among respondents who answered the item, with its JK_n standard error.

Level	n	Share
Secondary	940	0.582 (0.015)
None	38	0.024 (0.004)
Primary	326	0.202 (0.012)
Tertiary	312	0.193 (0.015)

Table 14: Segment prevalence within each level of education from Equation 18, with JK_n standard errors. Rows sum to one across segments.

Level	S1	S2	S3	S4	S5
Secondary	0.355 (0.015)	0.218 (0.015)	0.115 (0.011)	0.221 (0.014)	0.090 (0.009)
None	0.237 (0.071)	0.184 (0.063)	0.184 (0.064)	0.132 (0.054)	0.263 (0.075)
Primary	0.353 (0.030)	0.212 (0.027)	0.095 (0.017)	0.160 (0.021)	0.181 (0.022)
Tertiary	0.330 (0.029)	0.157 (0.022)	0.115 (0.018)	0.327 (0.031)	0.071 (0.016)



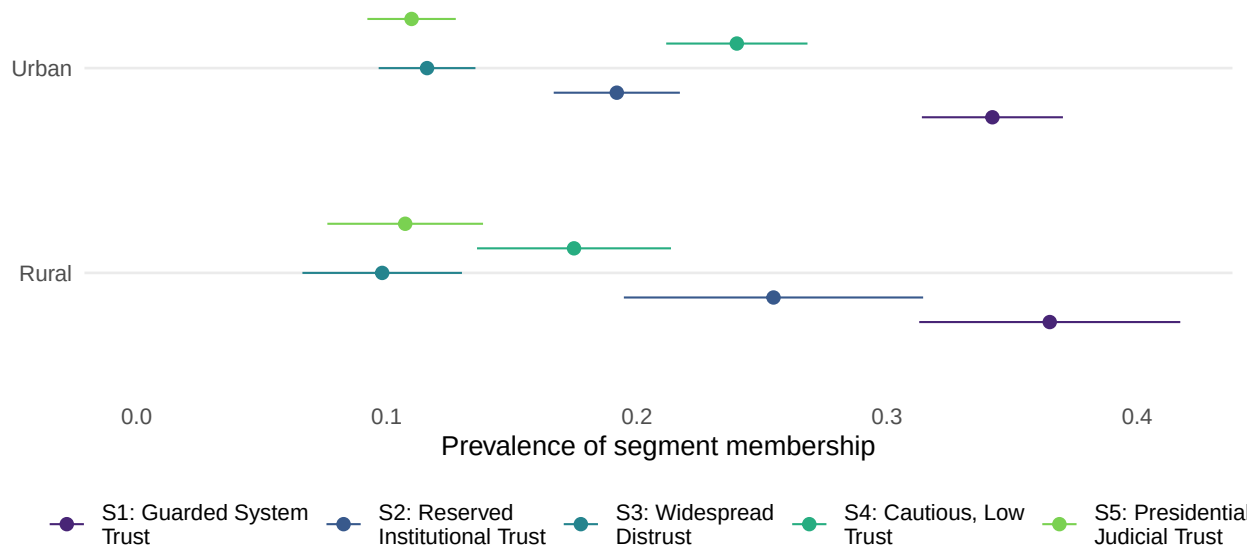
14.4 urban

Table 15: Composition by urban: design-weighted proportion among respondents who answered the item, with its JKn standard error.

Level	n	Share
Urban	1292	0.799 (0.035)
Rural	326	0.201 (0.035)

Table 16: Segment prevalence within each level of urban from Equation 18, with JKn standard errors. Rows sum to one across segments.

Level	S1	S2	S3	S4	S5
Urban	0.342 (0.014)	0.192 (0.013)	0.116 (0.010)	0.240 (0.014)	0.110 (0.009)
Rural	0.365 (0.026)	0.255 (0.030)	0.098 (0.016)	0.175 (0.020)	0.107 (0.016)



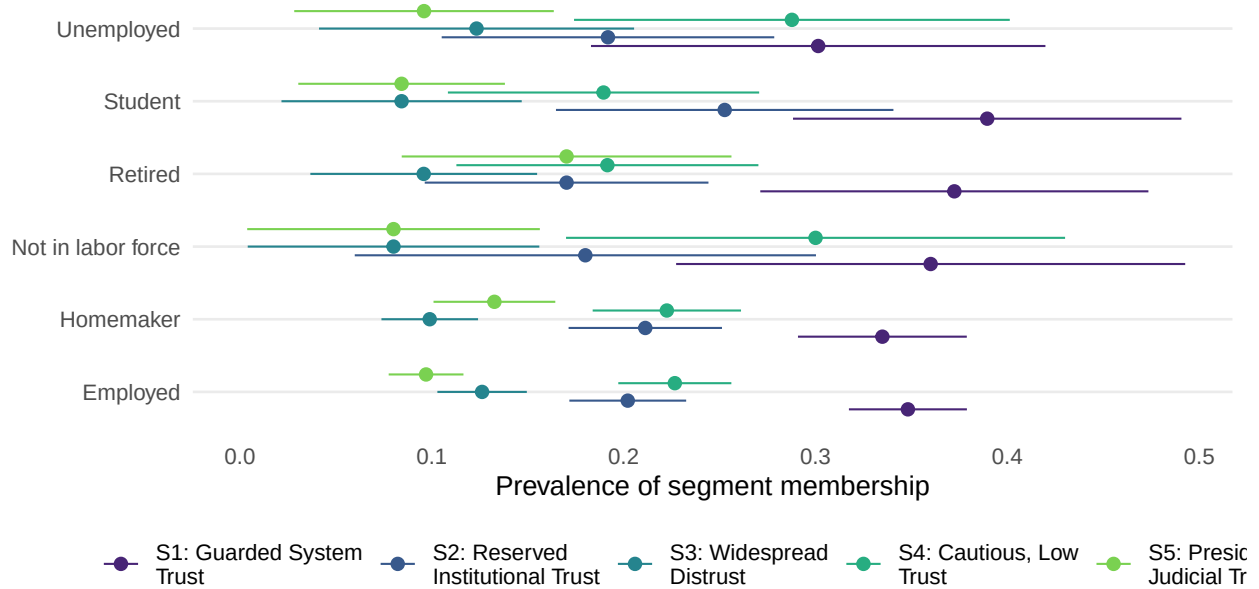
14.5 employment

Table 17: Composition by employment: design-weighted proportion among respondents who answered the item, with its JKn standard error.

Level	n	Share
Employed	856	0.531 (0.012)
Unemployed	73	0.045 (0.005)
Not in labor force	50	0.031 (0.004)
Student	95	0.059 (0.006)
Homemaker	445	0.276 (0.012)
Retired	94	0.058 (0.006)

Table 18: Segment prevalence within each level of employment from Equation 18, with JKn standard errors. Rows sum to one across segments.

Level	S1	S2	S3	S4	S5
Employed	0.348 (0.016)	0.202 (0.015)	0.126 (0.012)	0.227 (0.015)	0.097 (0.010)
Unemployed	0.301 (0.060)	0.192 (0.044)	0.123 (0.041)	0.288 (0.057)	0.096 (0.034)
Not in labor force	0.360 (0.067)	0.180 (0.061)	0.080 (0.038)	0.300 (0.066)	0.080 (0.039)
Student	0.389 (0.051)	0.253 (0.044)	0.084 (0.032)	0.189 (0.041)	0.084 (0.027)
Homemaker	0.335 (0.022)	0.211 (0.020)	0.099 (0.013)	0.222 (0.020)	0.133 (0.016)
Retired	0.372 (0.051)	0.170 (0.037)	0.096 (0.030)	0.191 (0.040)	0.170 (0.043)



15 The delivered data product

The workflow writes a labelled SPSS file carrying the segment assignment with value labels, the label text, the posterior probabilities, and the assignment diagnostics, alongside CSV exports of the measurement model with intervals and the domain estimates.

The posterior columns are exported deliberately. A downstream analyst who cross-tabulates the modal assignment alone reintroduces exactly the attenuation described in Section 14, and the variable label on the segment column says so.

```
out_sav <- survey_scored |>
  mutate(segment_label = if_else(is.na(segment), NA_character_,
                                segment_labels$Label[segment]),
         segment = haven::labelled(
           segment,
           labels = set_names(segment_labels$K, segment_labels$Label),
           label = paste("Segment (design-weighted LCA). Modal assignment:",
                         "cross-tabs on this variable alone are attenuated by",
                         "classification error. Use the post_segment columns for",
                         "uncertainty-aware analysis.")))

haven::write_sav(out_sav, file.path(cfg$out_dir, "survey_with_segments.sav"))

write_csv(mutate(ci_tbl, segment_label = segment_labels$Label[segment]),
          file.path(cfg$out_dir, "measurement_model_with_CIs.csv"))
write_csv(transmute(domain, demographic = variable, level, segment,
                    segment_label = segment_labels$Label[segment],
                    prevalence = p, se, lo, hi),
          file.path(cfg$out_dir, "segment_demographics.csv"))

saveRDS(list(fit_w = fit_w, fit_unit = fit_unit, pl_aligned = pl_aligned,
             validation = validation, enum = enum, ci_tbl = ci_tbl,
             bvr_tbl = bvr_tbl, item_discrim = item_discrim, domain = domain,
             dat = dat, post_w = post_w, modal_int = modal_int,
             w_vec = w_vec, K = K_star, nu = nu),
         file.path(cfg$out_dir, "model_fit.rds"))

cat(sprintf("Wrote survey_with_segments.sav (%d scored), two CSVs, and model_fit.rds to %s\n",
            nrow(scored), cfg$out_dir))
```

Wrote survey_with_segments.sav (1618 scored), two CSVs, and model_fit.rds to D:/repos/weighted.

16 Limitations

The pseudo-likelihood can be multimodal with near-tied optima. The reported solution is the best mode found under many seeded starts, and the JKn variance expresses within-mode design variance

only, not uncertainty about mode selection.

The measurement model is fit on item-complete cases while scoring and domain estimation extend to partially observed respondents. Respondents scored from fewer items carry more classification error, which is the correlated-error case flagged in Section 2: a processing error whose magnitude depends on an item nonresponse error.

Domain estimates use hard modal assignment and are attenuated. Differences between demographic levels are understated, so a difference that appears is real while a null is not evidence of no association.

Bivariate residuals and the indicator screen are rankings without reference distributions. They order item pairs and items by how poorly the model accounts for them and support no test.

The replicate count equals the number of PSUs, so the replicate covariance matrix has rank at most that number. Individual variances and pairwise covariances remain consistent, which is all the reported quantities require, but a joint Wald test across more dimensions than there are replicates is not available.

Where a design imposes quotas at the final selection stage, variables controlled by those quotas have suppressed between-cluster variance and standard errors that reflect fieldwork control rather than sampling information. Such variables' apparent precision should not be reported without comment.

Language model labels are drafts with an audited override path. No claim is made about their validity, and no reliability estimate across runs is reported.

17 Replication

Four files. `survey_1ca_source.R` is the engine and is never edited per dataset. `survey_data_config.R` is the only file edited per dataset. This document produces the analysis. A companion script audits object flow through the document in chunk order.

Six things change in a port.

The file path and item list. `item_codes` is a named character vector mapping analysis names to source variable names; the names become the analysis names automatically.

The nonresponse codes. `na_codes` lists the values representing nonresponse. The source file must be read with `user_na = TRUE`, since the default converts declared user-missing values to NA on import and the codes then never arrive.

The design variables. The configuration renames them to fixed names `id`, `strata`, `psu`, `wt`, so nothing downstream varies by dataset. Determining which source variables these are is the highest-consequence decision in a port: if the stratification variable is wrong, every standard error in this document is wrong and nothing else will reveal it.

The demographic recodes. One `recode_values` per variable, every observed source label named explicitly, with `unmatched = "error"` and an NA ~ NA arm. A regular expression scheme is not acceptable: a pattern that matches a label it was not intended to match silently misclassifies respondents, and one that matches nothing silently deletes them.

The profiling variable list. `cfg$aux` selects which demographics drive Section 14. Attitudinal variables should be excluded when they are plausibly correlated with the measurement items by construction, since profiling segments on them partly explains the measurement model with itself.

The configuration block. `K_range`, `n_starts`, `seed`, the information floor `min_items`, the complete-case switch, the output directory, and the language model endpoint.

The following halt the render rather than produce a wrong number: any demographic source label unmatched by its `recode_values` arms; any configured column absent from the data; any stratum containing a single PSU in the analysis frame, reported by stratum; a mismatch between the number of estimated domain quantities and their metadata; and a label file whose row count does not match K .

The following must be checked by the analyst: the recode audit, once per dataset; the weight coefficient of variation and unequal weighting effect, which determine whether weighting is doing anything in this file; the enumeration table and profiles, to choose K ; assignment quality by number of items answered, to set the information floor; and the drafted labels against their profile panels.

References

- Asparouhov, Tihomir. 2005. “Sampling Weights in Latent Variable Modeling.” *Structural Equation Modeling* 12 (3): 411–34. https://doi.org/10.1207/s15328007sem1203_4.
- Bakk, Zsuzsa, Daniel L. Oberski, and Jeroen K. Vermunt. 2014. “Relating Latent Class Assignments to External Variables: Standard Errors for Correct Inference.” *Political Analysis* 22 (4): 520–40. <https://doi.org/10.1093/pan/mpu003>.
- Bakk, Zsuzsa, Fetene B. Tekle, and Jeroen K. Vermunt. 2013. “Estimating the Association Between Latent Class Membership and External Variables Using Bias-Adjusted Three-Step Approaches.” *Sociological Methodology* 43 (1): 272–311. <https://doi.org/10.1177/0081175012470644>.
- Binder, David A. 1983. “On the Variances of Asymptotically Normal Estimators from Complex Surveys.” *International Statistical Review* 51 (3): 279–92. <https://doi.org/10.2307/1402588>.
- Bolck, Annabel, Marcel Croon, and Jacques Hagenaars. 2004. “Estimating Latent Structure Models with Categorical Variables: One-Step Versus Three-Step Estimators.” *Political Analysis* 12 (1): 3–27. <https://doi.org/10.1093/pan/mpu001>.
- Brackstone, Gordon. 1999. “Managing Data Quality in a Statistical Agency.” *Survey Methodology* 25 (2): 139–49.
- Dempster, Arthur P., Nan M. Laird, and Donald B. Rubin. 1977. “Maximum Likelihood from Incomplete Data via the EM Algorithm.” *Journal of the Royal Statistical Society, Series B* 39 (1): 1–38.
- Gilardi, Fabrizio, Meysam Alizadeh, and Maël Kubli. 2023. “ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks.” *Proceedings of the National Academy of Sciences* 120 (30): e2305016120. <https://doi.org/10.1073/pnas.2305016120>.
- Goodman, Leo A. 1974. “Exploratory Latent Structure Analysis Using Both Identifiable and Unidentifiable Models.” *Biometrika* 61 (2): 215–31. <https://doi.org/10.1093/biomet/61.2.215>.
- Groves, Robert M., Floyd J. Fowler, Mick P. Couper, James M. Lepkowski, Eleanor Singer, and Roger Tourangeau. 2009. *Survey Methodology*. 2nd ed. Hoboken, NJ: Wiley.
- Groves, Robert M., and Lars Lyberg. 2010. “Total Survey Error: Past, Present, and Future.” *Public Opinion Quarterly* 74 (5): 849–79. <https://doi.org/10.1093/poq/nfq065>.

- Hornik, Kurt. 2005. "A CLUE for CLUster Ensembles." *Journal of Statistical Software* 14 (12): 1–25. <https://doi.org/10.18637/jss.v014.i12>.
- Kish, Leslie. 1965. *Survey Sampling*. New York: Wiley.
- Kuhn, Harold W. 1955. "The Hungarian Method for the Assignment Problem." *Naval Research Logistics Quarterly* 2 (1–2): 83–97. <https://doi.org/10.1002/nav.3800020109>.
- LAPOP Lab. 2023. "AmericasBarometer 2023: Mexico. Technical Information and Country Dataset." Vanderbilt University; <https://www.vanderbilt.edu/lapop/>.
- Lazarsfeld, Paul F., and Neil W. Henry. 1968. *Latent Structure Analysis*. Boston: Houghton Mifflin.
- Linzer, Drew A., and Jeffrey B. Lewis. 2011. "poLCA: An R Package for Polytomous Variable Latent Class Analysis." *Journal of Statistical Software* 42 (10): 1–29. <https://doi.org/10.18637/jss.v042.i10>.
- Lumley, Thomas. 2004. "Analysis of Complex Survey Samples." *Journal of Statistical Software* 9 (8): 1–19. <https://doi.org/10.18637/jss.v009.i08>.
- Lumley, Thomas, and Alastair Scott. 2015. "AIC and BIC for Modeling with Complex Survey Data." *Journal of Survey Statistics and Methodology* 3 (1): 1–18. <https://doi.org/10.1093/jssam/smu021>.
- Nylund, Karen L., Tihomir Asparouhov, and Bengt O. Muthén. 2007. "Deciding on the Number of Classes in Latent Class Analysis and Growth Mixture Modeling: A Monte Carlo Simulation Study." *Structural Equation Modeling* 14 (4): 535–69. <https://doi.org/10.1080/10705510701575396>.
- Oberski, Daniel L., Geert H. van Kollenburg, and Jeroen K. Vermunt. 2013. "A Monte Carlo Evaluation of Three Methods to Detect Local Dependence in Binary Data Latent Class Models." *Advances in Data Analysis and Classification* 7 (3): 267–79. <https://doi.org/10.1007/s11634-013-0146-2>.
- Patterson, Blossom H., C. Mitchell Dayton, and Barry I. Graubard. 2002. "Latent Class Analysis of Complex Sample Survey Data: Application to Dietary Data." *Journal of the American Statistical Association* 97 (459): 721–28. <https://doi.org/10.1198/016214502388618465>.
- Ramaswamy, Venkatram, Wayne S. DeSarbo, David J. Reibstein, and William T. Robinson. 1993. "An Empirical Pooling Approach for Estimating Marketing Mix Elasticities with PIMS Data." *Marketing Science* 12 (1): 103–24. <https://doi.org/10.1287/mksc.12.1.103>.
- Rust, Keith F., and J. N. K. Rao. 1996. "Variance Estimation for Complex Surveys Using Replication Techniques." *Statistical Methods in Medical Research* 5 (3): 283–310. <https://doi.org/10.1177/096228029600500305>.
- Skinner, Chris J. 1989. "Domain Means, Regression and Multivariate Analysis." In *Analysis of Complex Surveys*, edited by C. J. Skinner, D. Holt, and T. M. F. Smith, 59–87. Chichester: Wiley.
- Vermunt, Jeroen K. 2010. "Latent Class Modeling with Covariates: Two Improved Three-Step Approaches." *Political Analysis* 18 (4): 450–69. <https://doi.org/10.1093/pan/mpq025>.
- Vermunt, Jeroen K., and Jay Magidson. 2007. "Latent Class Analysis with Sampling Weights: A Maximum-Likelihood Approach." *Sociological Methods and Research* 36 (1): 87–111. <https://doi.org/10.1177/0049124107301965>.
- Wolter, Kirk M. 2007. *Introduction to Variance Estimation*. 2nd ed. New York: Springer.
- Wu, Stephanie M., Matthew R. Williams, Terrance D. Savitsky, and Briana J. K. Stephenson. 2024. "Derivation of Outcome-Dependent Dietary Patterns for Low-Income Women Obtained from Survey Data Using a Supervised Weighted Overfitted Latent Class Analysis." *Biometrics* 80 (4): ujae122. <https://doi.org/10.1093/biometc/ujae122>.
- . 2026. "Baysc: An R Package for Bayesian Survey Clustering." *Journal of Open Source*

Software 11 (119): 8382. <https://doi.org/10.21105/joss.08382>.
Ziems, Caleb, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. “Can Large Language Models Transform Computational Social Science?” *Computational Linguistics* 50 (1): 237–91. https://doi.org/10.1162/coli_a_00502.